

La Programmation en langage C

Chapitre 7 : Les fonctions

La cible : MIP S3

Pr S.ELFILALI
sanaa.elfilali@univh2c.ma

La fonction main et printf

```
include <stdio.h>
```

```
int main()  
{  
    printf("message 1");  
    printf("message 2");  
    ...  
    return 0;  
}
```

```
void printf()  
{  
    .....  
}
```

C'est quoi une fonction ?

Une partie du code (programme), des blocs d'instructions que l'on peut ensuite réutiliser dans le code source en l'appelant par son nom

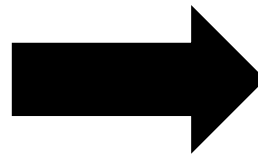
Utilisation des fonctions

```
int main()
{
    printf("Salut !");
    printf("Formation C");

    printf("Salut !");
    printf("Formation C");

    printf("Salut !");
    printf("Formation C");

    return 0;
}
```



```
int main()
{
    maFonction();
    maFonction();
    maFonction();
    return 0;
}
```

```
void maFonction()
{
    printf("Salut !");
    printf("Formation C");
}
```

Pourquoi utiliser les fonctions ?

Intérêt 1 : découper le programme en blocs pour pouvoir le réutiliser par la suite
Ensuite; mutualiser , factoriser pour éviter d'avoir plein de duplications de copier coller dans ce code source

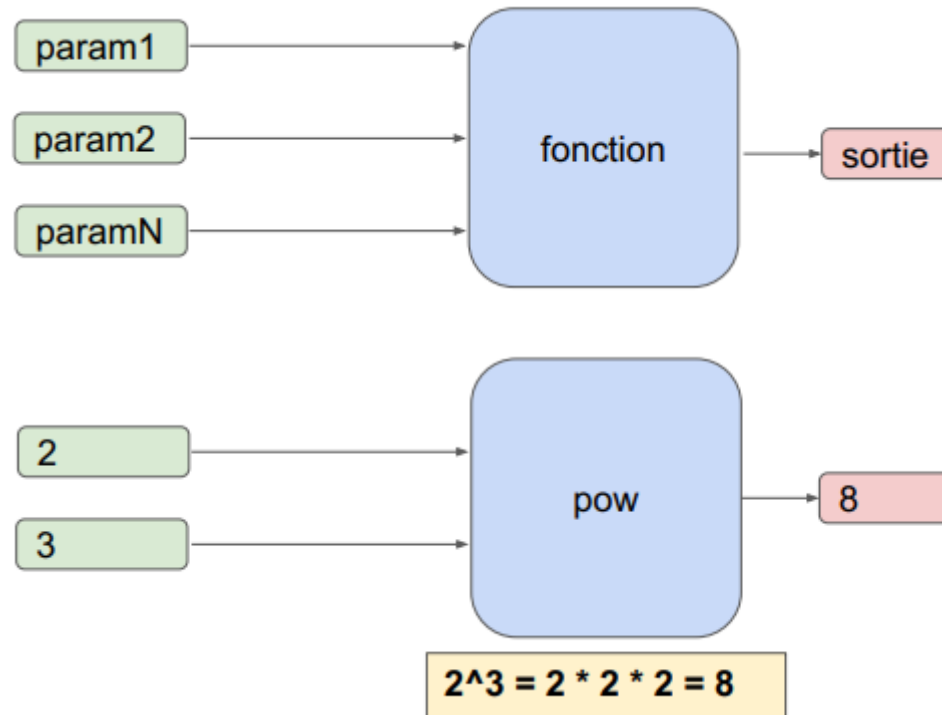
Intérêt 2 : découper le programme de manière logique pour pouvoir le rendre plus facile à comprendre par la suite

Utilisation des fonctions

```
int main()
{
    maFonction();
    maFonction();
    maFonction();
    return 0;
}
```

```
void maFonction()
{
    printf("Salut !");
    printf("Formation C");
}
```

Les fonctions



De quoi est composé une fonction ?

Un bloc d'instruction

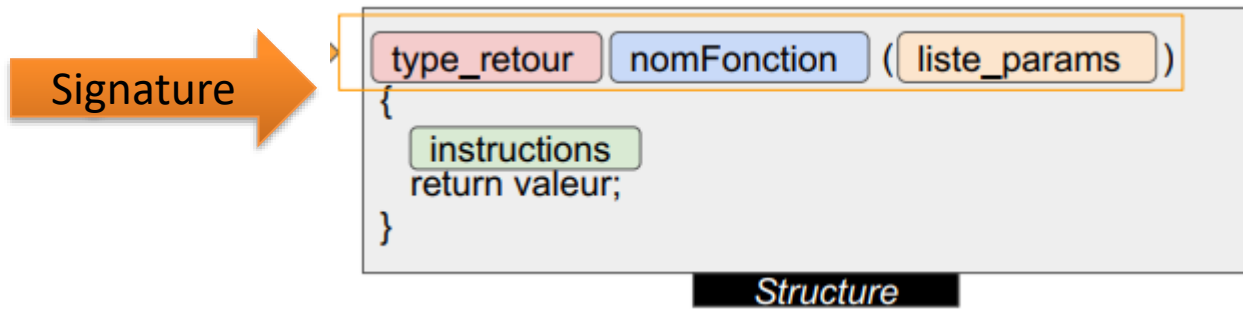
Un ensemble de paramètres en entrée qui permettent de varier les instructions de la fonction

Un résultat de sortie

Introduction

- Une fonction est un sous-programme qui accomplit une tâche spécifique.
- Elle possède une structure analogue à celle de la fonction main.
- L'utilisation de fonctions présente plusieurs avantages :
 - Rendre le programme plus facile à écrire et beaucoup plus facile à lire.
 - Eviter d'écrire plusieurs fois la même séquence de code.
 - La mise au point est plus simple car les différentes fonctions peuvent être testées séparément.
 - Elles peuvent être réutilisées par d'autres programmes.

Les fonctions



```
int main()
{
    return 0;
}
```

Exemple 1

```
void maFonction()
{
    ...
}
```

Exemple 2

```
void afficherAge(int age)
{
    printf("tu a %d ans\n", age);
    return;
}
```

Exemple 3

```
int somme(int var1, int var2)
{
    int total= var2 + var1;
    return total;
}
```

Exemple 4

Règles de nommage des fonctions

<u>Règles</u>
Pas de nombre en début de nom
Pas de caractères spéciaux à l'exception de _
Pas de mots réservés en c (ex type de variable)
Pas d'accent (à, é, è , ...)
Pas d'espace
Nom explicite (très important pour la lisibilité)
Minuscule sauf 1ere lettre mots séparateurs

<u>Exemples</u>
2fonction
nom-de-fonction
for
début
nom de fonction
nom_de_fonction
nomDeFonction

Les fonctions

Passer sur code::Blocks

Créer une fonction qui s appelle ditHello

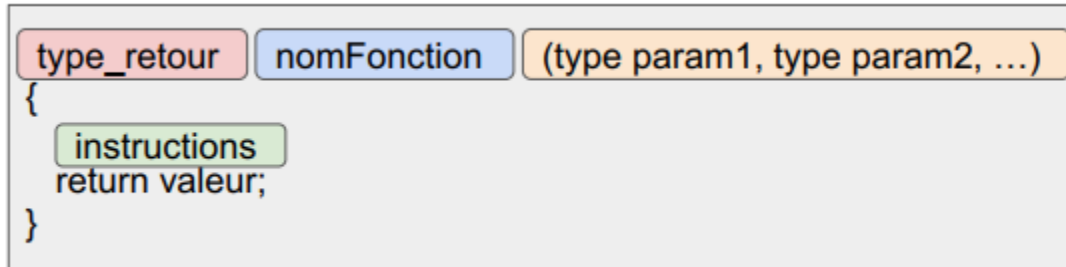
```
#include <stdio.h>
#include <stdlib.h>
void ditHello()
{
    printf("Hello world!\n");
    printf("cours du langage C \n");
}

int main()
{
    ditHello();
    ditHello();
    ditHello();

    return 0;
}
```

```
Hello world!
cours du langage C
Hello world!
cours du langage C
Hello world!
cours du langage C
```

Paramètres de fonction



```
void somme(int var1, int var2)  
{  
    int somme = var1+var2;  
    printf("%d+%d=%d", var1, var2, somme);  
}
```

```
int main()  
{  
    int ma_variable = 2;  
    somme(3,1);  
    somme(3, ma_variable);  
}
```

3+1=4
3+2=5

Paramètres de fonction

Passer sur code::Blocks

Créer une fonction qui va afficher des infos sur une personnage vidéo

```
void ditHello()
{
    printf("Hello world!\n");
    printf("cours du langage C \n");
}
void infoPerso(int xp, int lvl)
{
    printf("\txp= %d\n",xp);
    printf("\tlvl= %d\n",lvl);
}
int main()
{
    ditHello();
    printf("Mage :");
    infoPerso(1250,10);

    return 0;
}
```

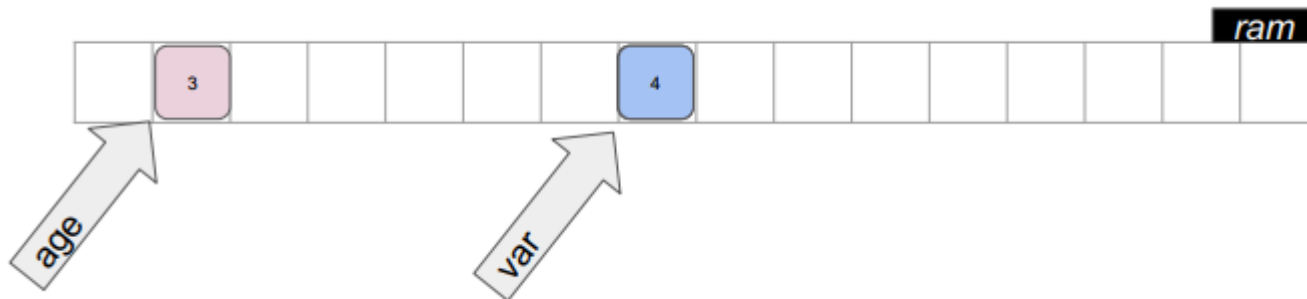
```
Hello world!
cours du langage C
Mage : xp= 1250
      lvl= 10
```

Passage par copie
Passage par valeur

Passage de paramètre par copie

```
int main()  
{  
    int age= 3;  
    anniversaire(age);  
}
```

```
void anniversaire(int var)  
{  
    var++;  
    printf("%d", var);  
}
```

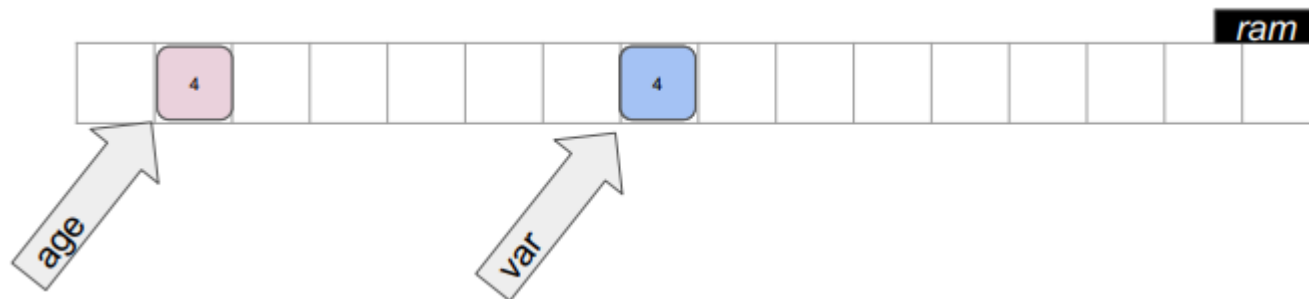


Passage par copie ou passage par valeur

Récupération du retour de fonction

```
int main()  
{  
    int age= 3;  
    age = anniversaire(age);  
}
```

```
int anniversaire(int var)  
{  
    var++;  
    return var;  
}
```



Passage de paramètre

Passer sur code::Blocks

Créer une fonction qui va afficher des infos sur un personnage vidéo

```
void ditHello()
{
    printf("Hello world!\n");
    printf("cours du langage C \n");
}
void infoPerso(int xp, int lvl)
{
    xp*=2;
    printf("\txp= %d\n",xp);
    printf("\tlvl= %d\n",lvl);
}
int main()
{
    int xp_mage=1250;
    ditHello();
    printf("Mage :");
    infoPerso(xp_mage,10);
    printf("\txp =%d\n",xp_mage);

    return 0;
}
```

```
Hello world!
cours du langage C
Mage : xp= 2500
      lvl= 10
      xp =1250
```

Les returns

```
void balrog()  
{  
    return;  
    printf("Vous ne passerez pas!");  
}  
  
int main()  
{  
    printf("Salut \n");  
    balrog();  
    return 0;  
}
```

Salut

Exemple

Les returns

Passer sur code::Blocks

Fonction qui calcule puissance (2^3)

```
#include <stdio.h>
#include <stdlib.h>
int puissance (int base, int exposant )
{
    int resultat ;
    int i;
    if (base==0 && exposant ==0)
    {
        return 0;
    }
    if (base==0 && exposant !=0)
    {
        return 0;
    }
    if (base!=0 && exposant ==0)
    {
        return 1;
    }
    resultat=1;
    for(i=0; i< exposant; i++)
    {
        resultat=resultat*base;
    }
    return resultat;
}
```

```
0^3=0
2^0=1
2^3=8
```

```
int main()
{

    printf("0^3=%d\n", puissance(0,3));
    printf("2^0=%d\n", puissance(2,0));
    printf("2^3=%d\n", puissance(2,3));
    return 0;
}
```

Exercice : lanceur de dés

1- Créer une fonction De, qui prend en entrée 1 entier qui correspond au nombre de faces d'un dé et qui retourne aléatoirement un numéro de face.

Exemple de(6) correspond à un lancé de dé de 6, donc un retour entre 1 et 6.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int de(int);
int main()
{
    // initialisation de la génération aléatoire
    srand(time(NULL));
    printf("on lance un De de %d :%d",6,de(6));
    return 0;
}
int de(int nb_faces)
{
    int resultat = (rand()% nb_faces)+1;
    return resultat;
}
```

1

on lance un De de 6 :6

Exercice : lanceur de dés

2- Créer une fonction Des, qui prend en entrée 2 entiers - Nombre de dés à lancer. - Nombre de faces par dé.

Cette fonction retourne alors la somme des résultats de chaque dé.

```
#include <stdio.h>#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int de(int);
int des(int, int);
int main()
{ // initialisation de la génération aléatoire
  srand(time(NULL));

  printf("on lance %d de Des de %d :%d\n",3,6,des(3,6));
  return 0;
}
int de(int nb_faces)
{
  int resultat = (rand()% nb_faces)+1;
  return resultat;
}
int des (int nb_des,int nb_faces)
{
  int i,No_face;
  int total=0;
  for(i=1;i<=nb_des;i++)
  {

    No_face=de(nb_faces);

    printf("\n la face du de numero %d = %d\n", i, No_face);
```

2

la face du de numero 1 = 2

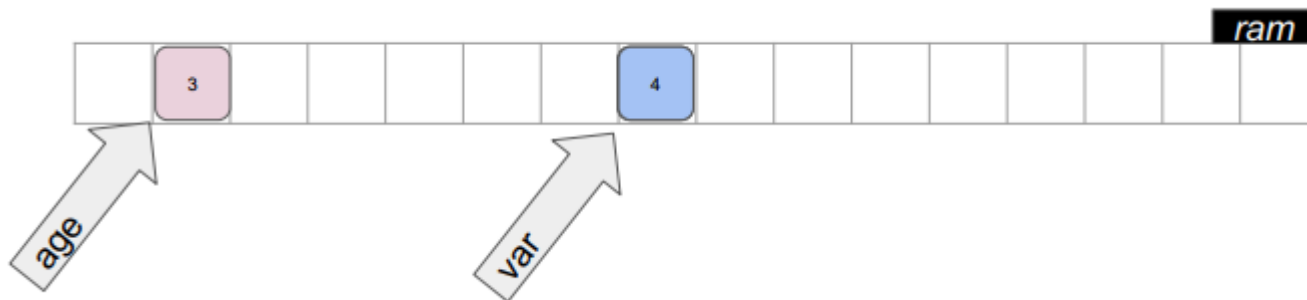
la face du de numero 2 = 1

la face du de numero 3 = 6
on lance 3 de Des de 6 :9

Passage de paramètre par copie

```
int main()  
{  
    int age= 3;  
    anniversaire(age);  
}
```

```
void anniversaire(int var)  
{  
    var++;  
    printf("%d", var);  
}
```

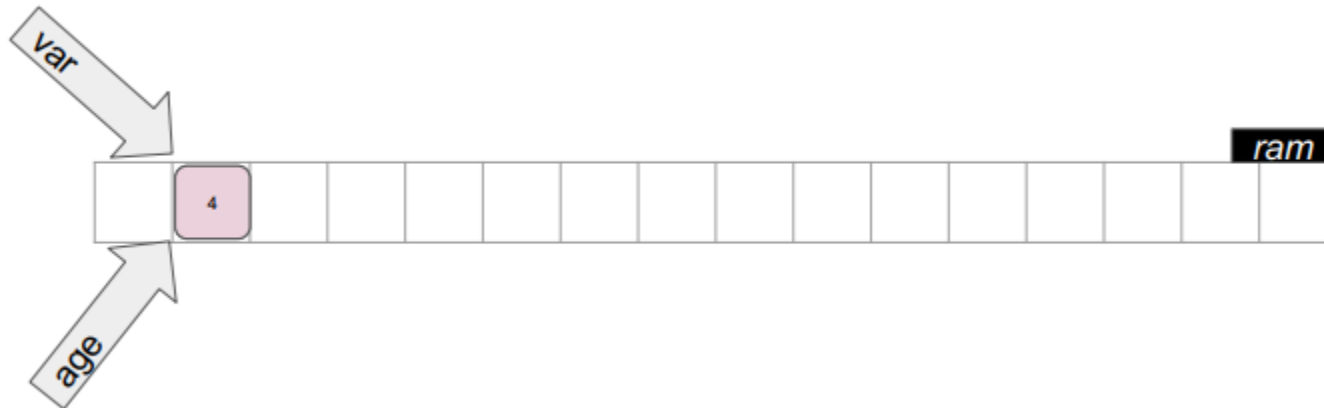


Passage par copie ou passage par valeur

Passage des paramètres par pointeur

```
int main()  
{  
    int age= 3;  
    anniversaire(&age);  
}
```

```
void anniversaire(int* var)  
{  
    *var++;  
    printf("%d", *var);  
}
```



Passage des paramètres par pointeur

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
void inverser(int nb1,int nb2)
{
    int tmp =nb2;
    nb2= nb1;
    nb1= tmp;
}
```

```
int main()
{
    int nombre_1=12;
    int nombre_2=89;
    printf("nombre_1 =%d\n",nombre_1);
    printf("nombre_2 =%d\n",nombre_2);
    inverser(nombre_1,nombre_2);
    printf("inverser :\n");
    printf("nombre_1 =%d\n",nombre_1);
    printf("nombre_2 =%d\n",nombre_2);

    return 0;
}
```

```
nombre_1 =12
nombre_2 =89
inverser :
nombre_1 =12
nombre_2 =89
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int inverser(int* nb1,int* nb2)
{
    int tmp ;
    if(nb1==NULL || nb2==NULL)
    {
        return -1;//sortir de la fonction
    }
    tmp =*nb1;
    *nb1= *nb2;
    *nb2= tmp;
    return 0;
}
```

```
int main()
{
    int nombre_1=12;
    int nombre_2=89;
    printf("nombre_1 =%d\n",nombre_1);
    printf("nombre_2 =%d\n",nombre_2);
    if(inverser(&nombre_1,&nombre_2) == 0 )
    {
        printf("inverser !\n");
    }
    printf("nombre_1 = %d\n", nombre_1);
    printf("nombre_2 = %d\n", nombre_2);
    return 0;
}
```

```
nombre_1 =12
nombre_2 =89
inverser !
nombre_1 = 89
nombre_2 = 12
```

Seance suivante

Test

- Donner l'affichage du programme suivant

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int i=10;
    while(i>0)
    {
        i=i-4;
        printf("%d    ", i);
    }
    return 0;
}
```

Le passage de tableau

Sur code::blocks, trouver le nombre max sur un tableau a une dimension

```
#include <stdio.h>
#include <stdlib.h>

int max1D(int tab[], int taille)
{
    int valeur_max = tab[0];
    int i;
    for(i=1;i<taille;i++)
    {
        if (valeur_max<tab[i])
            valeur_max=tab[i];
    }
    return valeur_max;
}

int main()
{
    int tab1D[5]={22,15,78,1,-6};
    int nombre_max_1D = max1D(tab1D,5);
    printf("le nombre le plus grand du tab1D est %d \n",nombre_max_1D);

    return 0;
}
```

le nombre le plus grand du tab1D est 78

Le passage de tableau

Sur code::blocks, trouver le nombre max sur un tableau a deux dimensions

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int max1D(int tab[], int taille)
{
    int valeur_max =0;
    int i;
    for(i=0;i<taille;i++)
    {
        if(tab[i]>valeur_max)
            valeur_max=tab[i];
    }
    return valeur_max;
}
```

```
int max2D(int tab[][5], int nb_ligne,int nb_colonne)
{
    int valeur_max =0;
    int i,j;
    for(i=0;i<nb_ligne;i++)
    {
        for(j=0;j<nb_colonne;j++)
        if(tab[i][j]>valeur_max)
        {
            valeur_max=tab[i][j];
        }
    }
    return valeur_max;
}
```

```
int main()
{
    int tab1D[5]={22,50,14,17,5};
    int tab2D[2][5]={{22,50,2,6,14},{2,5,2,6,94}};
    int nombre_max_1d=max1D(tab1D,5);
    int nombre_max_2d=max2D(tab2D,2,5);
    printf("le nombre le plus grand du tab1D est
%d\n",nombre_max_1d);
    printf("le nombre le plus grand du tab2D est
%d\n",nombre_max_2d);

    return 0;
}
```

le nombre le plus grand du tab1D est 50
le nombre le plus grand du tab2D est 94

Fonction et pointeur de tableau

```
#include <stdio.h>
#include <stdlib.h>

int max2D(int* tab, int nb_ligne, int nb_colonne)
{
    int valeur_max = *(tab);
    int i, j;
    int *p_ligne = NULL;
    for(i=0; i<nb_ligne; i++)
    {
        p_ligne = tab + (i * nb_colonne);

        for(j=0; j<nb_colonne; j++)

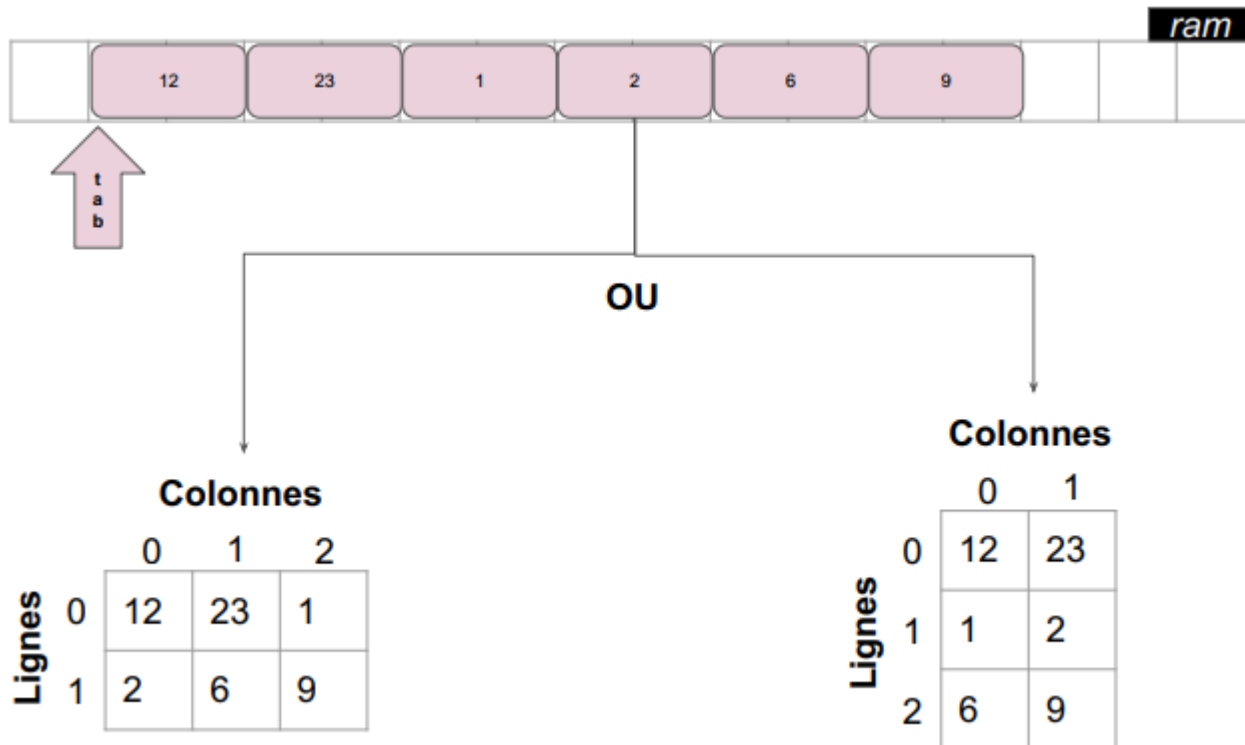
            if (valeur_max < *(p_ligne + j))
                valeur_max = *(p_ligne + j);
    }
    return valeur_max;
}

int main()
{
    int tab2D[2][5] = {{22, 15, 78, 1, -6}, {14, 18, 98, 11, 46}};
    int nombre_max_2D = max2D((int*)tab2D, 2, 5);

    printf("le nombre le plus grand du tab2D est %d \n", nombre_max_2D);
    return 0;
}
```

le nombre le plus grand du tab2D est 98

Structure d'un tableau



Prototype des fonctions

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int max1Dpointeur(int* tab, int taille); //int max1Dpointeur(int*, int );
int max2DPointeur(int* tab, int nb_ligne, int nb_colonne); //int
max2DPointeur(int* , int ,int);
```

```
int main()
{
    int tab1D[5]={22,50,14,17,5};
    int tab2D[2][5]={{22,50,2,6,14},{2,5,2,6,94}};
    int nombre_max_1d=max1Dpointeur(tab1D,5);
    int nombre_max_2d=max2DPointeur((int*)tab2D,2,5);
    printf("le nombre le plus grand du tab1D est %d\n",nombre_max_1d);
    printf("le nombre le plus grand du tab2D est %d\n",nombre_max_2d);

    return 0;
}
```

```
int max1Dpointeur(int* tab, int taille)
{
    int valeur_max =0;
    int i;
    for(i=0;i<taille;i++)
    {
        if(*(tab+i)>valeur_max)
            valeur_max=*(tab+i);
    }
    return valeur_max;
}
```

le nombre le plus grand du tab1D est 50
le nombre le plus grand du tab2D est 94

```
int max2DPointeur(int* tab, int nb_ligne, int nb_colonne)
{
    int valeur_max =0;
    int i,j;
    int *p_ligne=0;
    for(i=0;i<nb_ligne;i++)
    {
        p_ligne=tab+(i*nb_colonne);
        for(j=0;j<nb_colonne;j++)

            if(*(p_ligne+j)>valeur_max)
            {
                valeur_max= *(p_ligne+j);
            }
    }
    return valeur_max;
}
```

Prototype des fonctions

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int max1D(int [], int );
int max2D(int [][][5], int ,int );
int main()
{
    int tab1D[5]={22,50,14,17,5};
    int tab2D[2][5]={{22,50,2,6,14},{2,5,2,6,94}};
    int nombre_max_1d=max1D(tab1D,5);
    int nombre_max_2d=max2D(tab2D,2,5);
    printf("le nombre le plus grand du tab1D est
%d\n",nombre_max_1d);
    printf("le nombre le plus grand du tab2D est
%d\n",nombre_max_2d);

    return 0;
}
```

```
int max1D(int tab[], int taille)
{
    int valeur_max =0;
    int i;
    for(i=0;i<taille;i++)
    {
        if(tab[i]>valeur_max)
            valeur_max=tab[i];
    }
    return valeur_max;
}
```

```
int max2D(int tab[][][5], int nb_ligne,int nb_colonne)
{
    int valeur_max =0;
    int i,j;
    for(i=0;i<nb_ligne;i++)
    {
        for(j=0;j<nb_colonne;j++)
        {
            if(tab[i][j]>valeur_max)
            {
                valeur_max=tab[i][j];
            }
        }
    }
    return valeur_max;
}
```

le nombre le plus grand du tab1D est 50
le nombre le plus grand du tab2D est 94

Quizz kahoot

Devoir a rendre : jeu de Morpion

Nous allons créer un petit jeu de type Morpion. L'objectif est d'aligner 3 symboles identiques dans une grille de 3X3. Le jeu se joue à deux. Chaque joueur possède un symbole parmi 'X' et 'O'. A tour de rôle, ils dessinent dans un cas libre de la grille leur symbole respectif. Celui qui aligne 3 de ses symboles (horizontal, vertical ou diagonal), remporte la victoire. En cas de grille pleine sans alignement de 3 symboles identiques, c'est un match nul !

```
...
|X|O|X|
|X|O|O|
|. |X|O|
Joueur X ou voulez-vous jouer (x y)? 1 1
Case deja occupee!
Joueur X ou voulez-vous jouer (x y)? 2 0
Victoire du joueur X
|X|O|X|
|X|O|O|
|X|X|O|
```

FIN

Seance de revision

- Travailler deux examens
- A envoyer au delegué